

National Tsing Hua University

Department of Electrical Engineering

EE429200 IC Design Laboratory, Fall 2025

Homework Assignment #5 (15%, total 100 points)

Digit classification using Transformer

Assigned on Nov 6, 2025

Due by Nov 27, 2025

Introduction

Vision Transformers (ViTs) have emerged as a powerful alternative to traditional convolutional approaches for computer vision tasks. The pioneering work on ViT [1] demonstrated that, with sufficient data and computational resources, transformer-based models can achieve state-of-the-art performance on image classification tasks, opening the door to a new era of vision architecture beyond CNNs.

In this assignment, we will implement a multi-head attention of a transformer block. The implementation will also include linear projection, softmax and normalization operations.

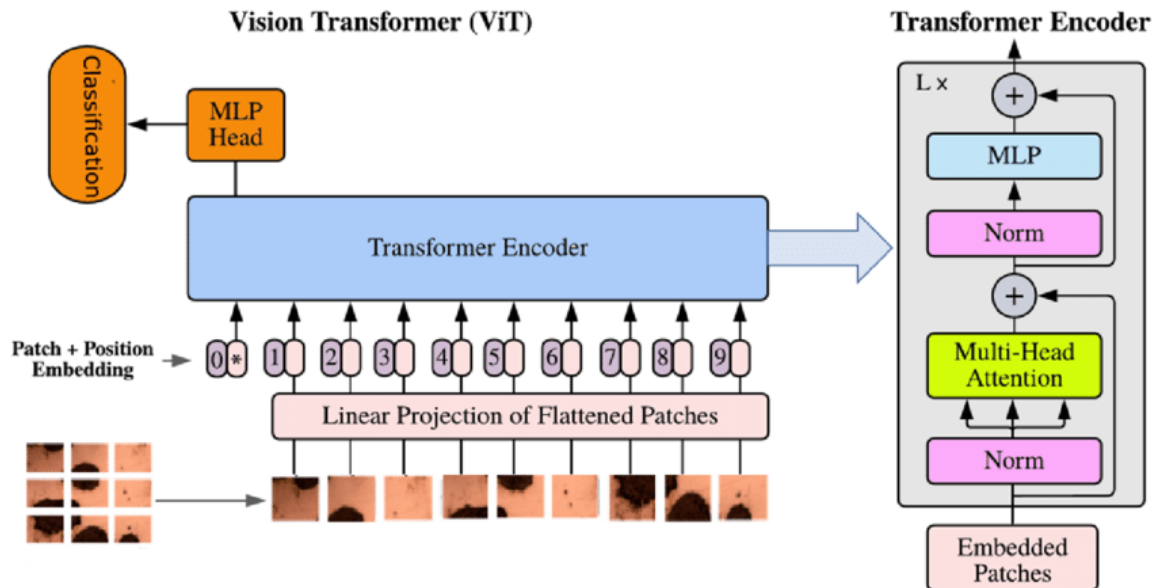


Fig. 1 Reference Model Architecture

Assignment Description

In this assignment, we provide four single-port SRAM groups, each group has 4 banks: SRAM_A (A0~A3), SRAM_B (B0~B3), SRAM_C (C0~C3) and SRAM_D(D0~D3) for you to access feature maps of each layer. Another two read-only SRAMs are provided for you to access parameters (for weights and biases). Since we have implemented the patched embedding and position encoding layer by testbench, you could load the prepared feature maps from SRAM_A at the beginning. Apply multi-head self-attention operation and write feature maps to SRAM_B. Please Follow the data flow illustrated in Fig.2 to finish the inference. More details will be mentioned in the **Hardware Design** Section. **Note that this dataflow is provided for reference only, and you may modify it to improve performance. However, when verifying your results, ensure that all outputs are written back in the correct order. For example, if you only complete the MHSA modules, make sure to store the results correctly back to SRAM D.**

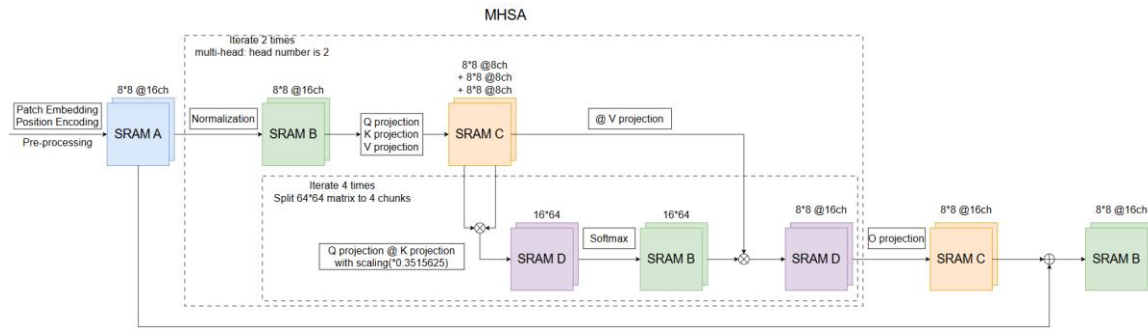


Fig. 2 Referenced Dataflow

Algorithm

Patch Embedding and Position Encoding (Pre-processed)

In this assignment, the input image ($32 \times 32 @ 3$ channels) is divided into 4×4 patches, giving 64 patches in total. Each patch ($4 \times 4 \times 3 = 48$ values) is flattened into a vector and passed through a linear layer that projects it to 16 channels. This process, called patch embedding, converts the image into 64 tokens, each with 16 channels, so the model can process them like a sequence in a transformer.

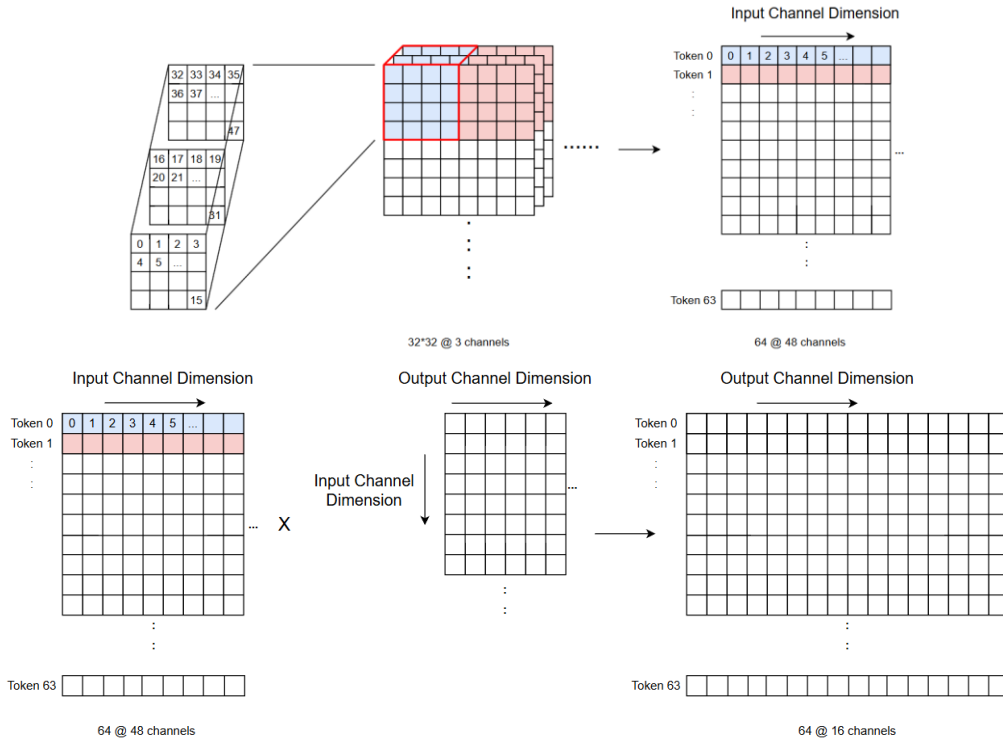


Fig. 3 Patch Embedding

In Vision Transformers (ViT), position encoding is used to give the model information about the spatial location of each image patch, since transformers themselves have no sense of order. After obtaining patch embeddings, a positional embedding vector (learned during training) is added to each token embedding elementwise. This preserves the same dimensions while allowing the model to distinguish where each patch comes from in the original image.

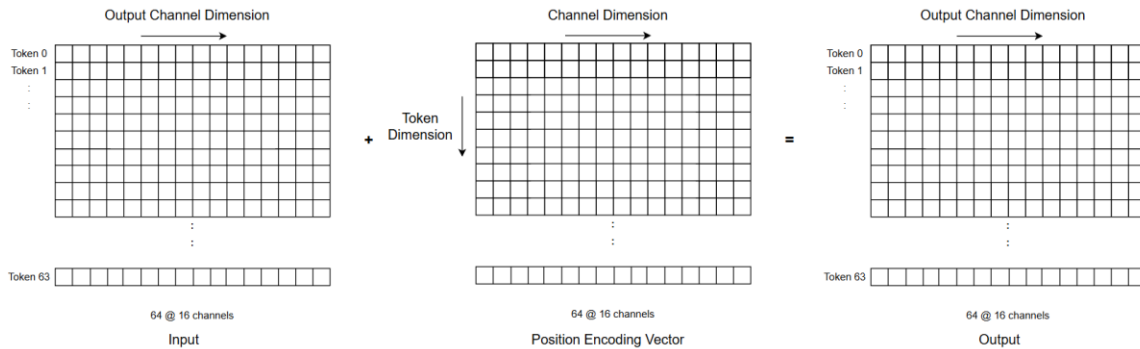


Fig. 4 Position Encoding

Normalization

Normalization is a technique to stabilize training in deep networks like ViT (Vision Transformer). It makes sure the input to each layer has a consistent scale and distribution, which helps learning. In this assignment, we apply L1 layer normalization for simplicity [2]. Suppose a token has c channels: $x = [x_0, x_1, x_2, \dots, x_{c-1}]$. First, we can compute means across channels: $\mu = \frac{1}{c} \sum_{i=0}^{c-1} x_i$ and mean of absolute error across channels $MAE = \frac{1}{c} \sum_{i=0}^{c-1} |x_i - \mu|$. Subsequently, each channel is normalized using the calculated statistics where the normalized token $\hat{x}_i$ is defined by the following operation:

$$\hat{x}_i = \frac{x_i - \mu}{MAE}$$

Finally, a channel-wise affine transformation is applied, where the learnable parameters weight: $\gamma = [\gamma_0, \gamma_1, \gamma_2, \dots, \gamma_{c-1}]$ and bias: $\beta = [\beta_0, \beta_1, \beta_2, \dots, \beta_{c-1}]$.

$$y_i = \gamma_i * \hat{x}_i + \beta_i$$

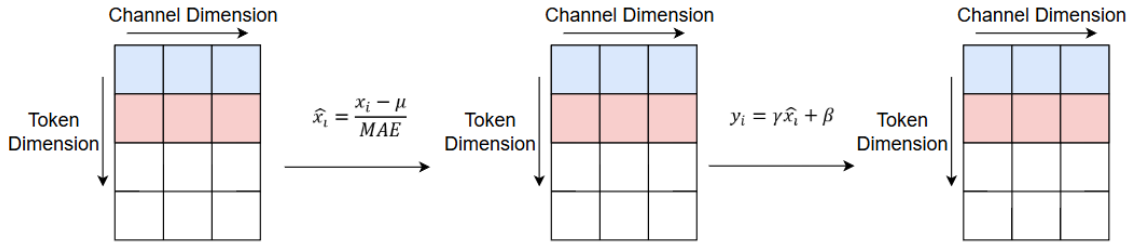


Fig. 5 Normalization

Linear Projection Layer

In a Transformer (including ViT), before doing attention, we need to transform the input features into four new spaces: Q (Query), K (Key), V (Value) and O (Output, after attention). To do this, we just need a simple matrix–vector multiplication:

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V$$

Here, X represents input tokens (each token has c_{in} channels), while W_Q, W_K, W_V are weight matrices of size: $c_{in} \times c_{out}$. The resulting Q, K , and V are output tokens with c_{out} channels. Fig. 6 shows an example of a linear projection that maps a 4-channel input to an 8-channel output. In this assignment, the input feature map is pre-processed into **64 tokens**, each with **16 channels**, and the **Q, K, V and O projections** map each token into another **16-channel token**. Each token is multiplied by the same weight matrix to complete the linear projection layer.

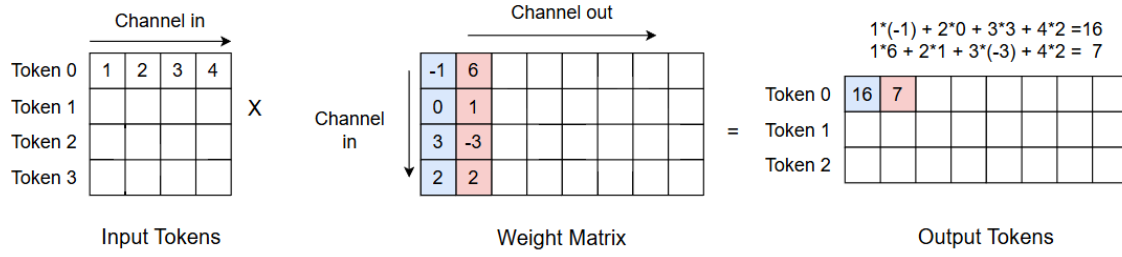


Fig. 6 Linear Projection

Multi-head Self-attention

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{c_{head}}}\right)V$$

Self-attention is the core operation of the Transformer and Vision Transformer (ViT) models. It allows the model to decide which parts of the input are most relevant when producing each output. Each input token is first projected into three vectors — Query (Q), Key (K), and Value (V) all with dimension c . The model computes the similarity between every query and all keys using the dot product QK^T . This produces a score matrix that measures how much each token should “attend to” others. To keep the scale stable, the scores are divided by $\sqrt{c_{head}}$, the square root of the dimension of each head. Then, a Softmax function is applied to normalize these scores into attention weights that sum to one. Finally, these weights are used to compute a weighted sum of the Value (V) vectors — giving the output of the attention layer. A simple example is shown below:

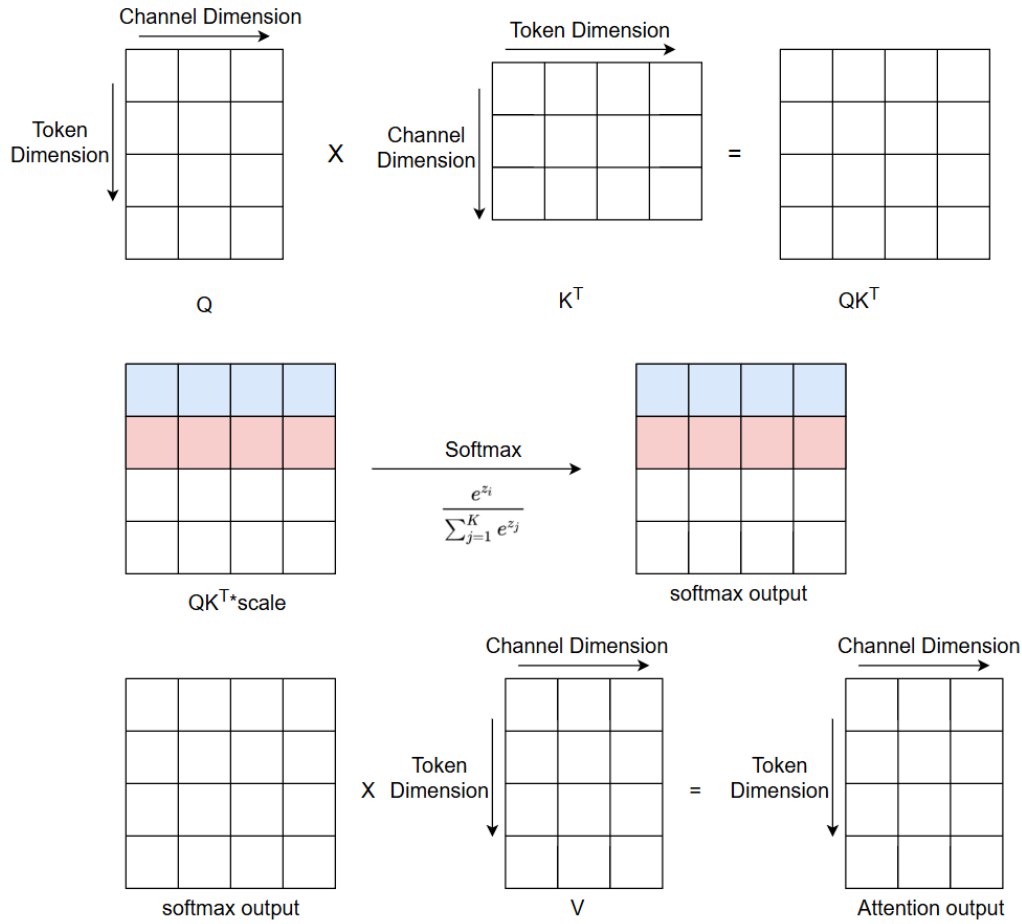


Fig. 7 Self-Attention

Multi-head attention means performing the attention operation in parallel across multiple heads. For example, if the number of heads is set to 2, Q , K , and V are each split into two parts—the first half of the channels form head 0, and the second half form head 1. Each head undergoes its own attention process, producing two separate attention outputs. Finally, these outputs are concatenated along the channel dimension to obtain the final attention result. Fig. 8 show a simple process of Multi-head Self-Attention. In this assignment, we set the number of head to 2 and $c = 16, \sqrt{c_{head}} = \sqrt{8}$.

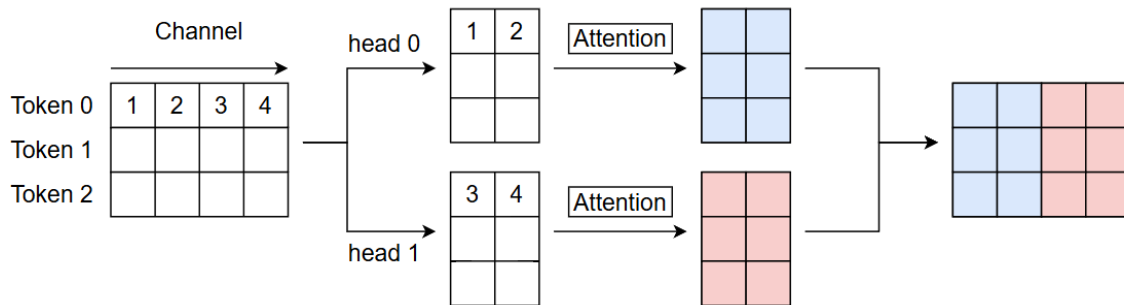


Fig. 8 Multi-head Self-Attention

Softmax

Softmax is a mathematical function widely used in machine learning, particularly for classification tasks. The softmax function takes a K-dimensional vector as input and compresses it into another K-dimensional vector, where each element falls within the range of (0, 1), and the sum of all elements equals 1. This property allows the output to be interpreted as probability. The formula for the softmax function is given by:

$$y_i = \frac{e^{x_i}}{\sum_{n=1}^K e^{x_n}} \text{ for } i = 1, \dots, K$$

In this equation, x represents the input features, and y represents the output probabilities. An example of softmax activation function is shown Fig. 12.

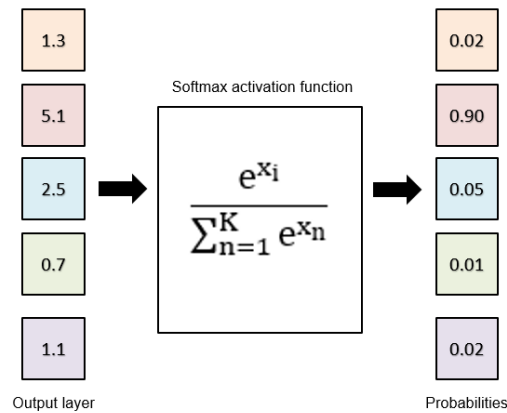


Fig. 9 Softmax activation

Residue Connection

Residue connection is a technique to solve the gradient vanishing problem in the deeper model. After we finish the multi-head self-attention and O projection, perform element-wise addition on the output and residue. Fig. 10 shows the details of the residual connection.

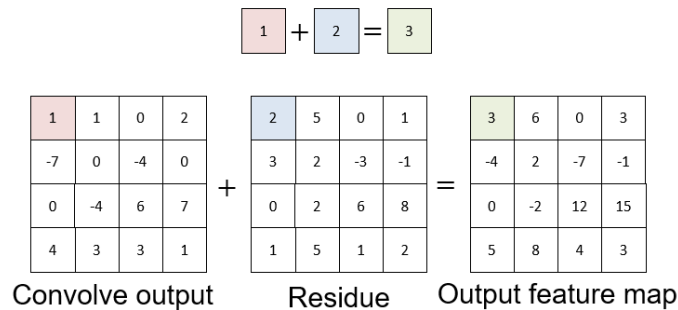


Fig. 10 example of 4X4 feature map with residue

Hardware Design

Activation SRAM Mapping

SRAM Group A consists of four banks (A0–A3), each with 32 addresses and a data width of 80 bits. SRAM Groups B, C, and D share the same structure as Group A but contain 32, 48, and 64 addresses, respectively.

Each address of SRAM can store 80 bits, as each channel of a token is represented by 10 bits. Therefore, one address can hold 8 channels of a token ($8 \times 10 = 80$ bits). To enable parallel access, four banks are interleaved, allowing four addresses to be sent to four banks simultaneously. By accessing all four banks in parallel, the system can read or write a $2 \times 2 @ 8\text{ch}$ token in a single cycle.

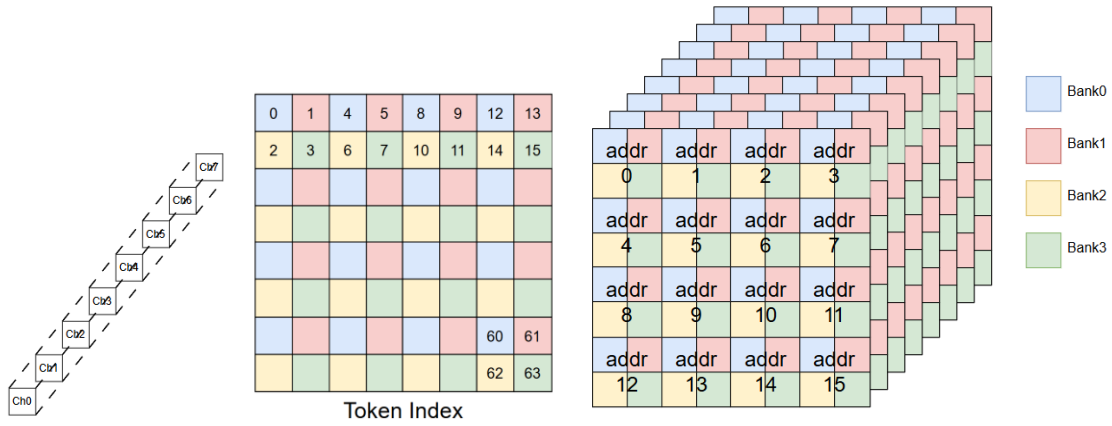


Fig. 11 SRAM data ordering in one bank and SRAM bank illustration

With the concept of SRAM group, now, we address the details of how we store output feature maps of each layer to the SRAM group by mapping the feature maps to specific addresses. Since the SRAM group can only store 8 channels in one address, layers with a larger number of channels, they need to partition their feature maps along channel dimensions and store them in different addresses. For the Q, K, and V projections, the results are stored in SRAM Group C: addresses 0~15 for Q, 16~31 for K, and 32~48 for V. Since the attention layer output shape is 64×64 (larger than our SRAM group), we should separate them by chunks and calculate sequentially. More details will be mentioned in the **Mutli-head self-attention implementation** Section.

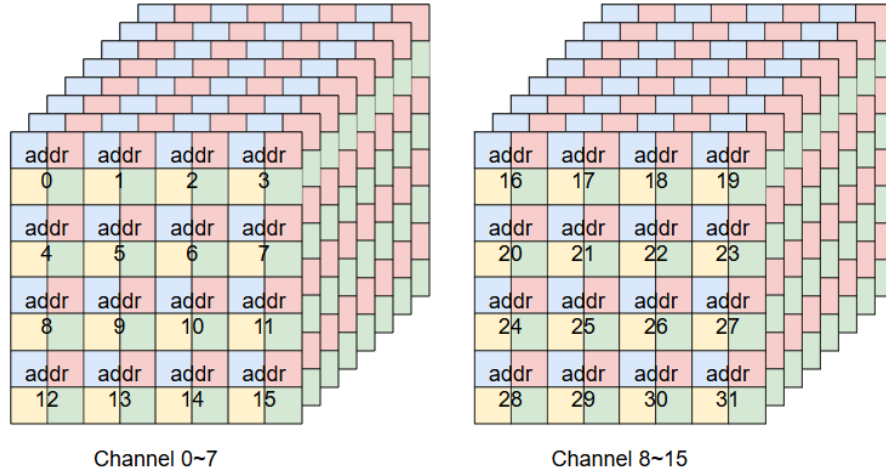


Fig. 12 4-bank SRAM mapping for input, normalization layer, O projection and residual add layer

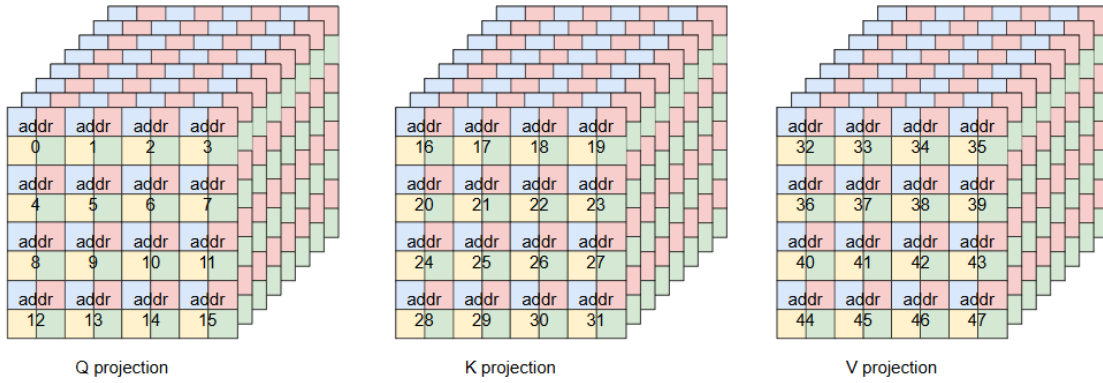


Fig. 13 4-bank SRAM mapping for QKV projection layer

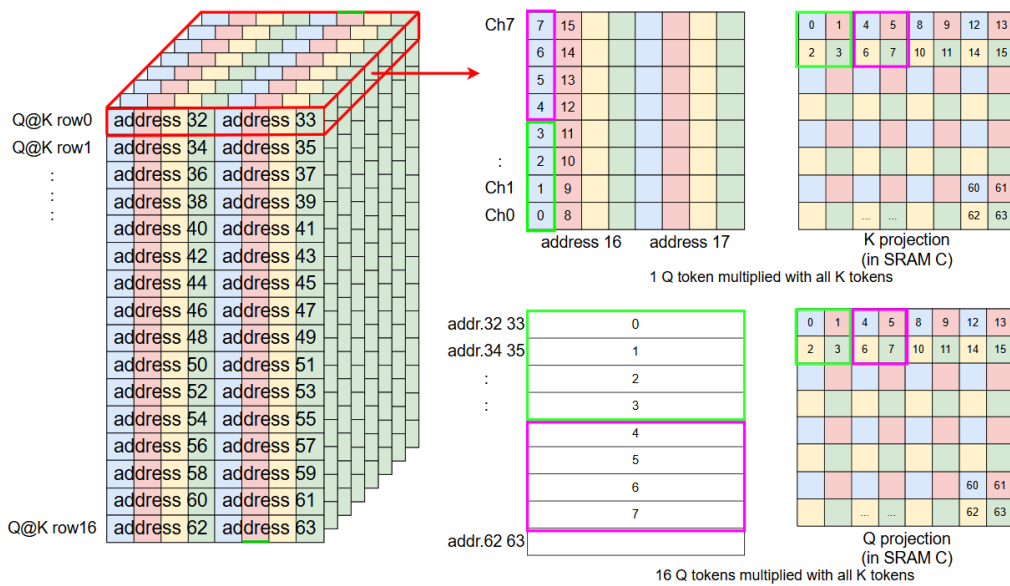


Fig. 14 4-bank SRAM mapping for Q@K and Softmax layer

Parameter SRAM Mapping

We use 10 bits to represent the value of model parameters. Another two SRAMs are used to store them, one is for weights, and the other is for biases. For weights, we store 16 weights into one address. For normalization layer, we store $\gamma = [\gamma_0, \gamma_1, \gamma_2 \dots \gamma_{15}]$ into one address. For linear projection, we store each column of 16×16 weight matrix in one address. For biases, we store one bias in one address.

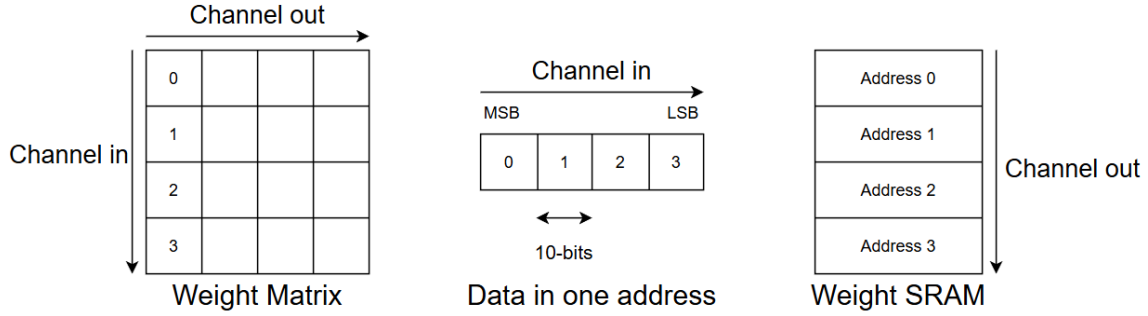


Fig. 15 Weight SRAM mapping for linear projection weight matrix

Weight SRAM		Bais SRAM	
ADDR	DATA	ADDR	DATA
0	norm wieght: $\{\{ch0, ch1 \dots ch15\}\}$	0	norm bias for ch0
1	Q projection matrix col 0	1	norm bias for ch1
2	Q projection matrix col 1	$\vdots$	norm bias for ch2~14
$\vdots$	Q projection matrix col 2~15	15	norm bias for ch15
17	K projection matrix col 0		
$\vdots$	K projection matrix col 1~15		
33	V projection matrix col 0		
$\vdots$	V projection matrix col 1~15		
49	O projection matrix col 0		
$\vdots$	O projection matrix col 1~14		
64	O projection matrix col 15		

Fig. 16 parameter SRAM mapping

Quantization

32-bit floating point is commonly used in GPUs and CPUs, and it has a wide dynamic range which is beneficial for accuracy. In ASIC design, we typically use fixed-point to represent numbers instead of floating points. In this assignment, we use signed fixed-point numbers in 2's complement format, consisting of one sign bit, an integer part, and a fractional part. Fig. 17 is an example of 12-bit fixed-point number with fractional length (bit-width of fractional part) equal to 4.

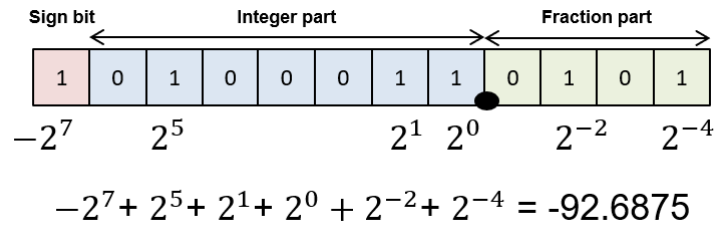


Fig. 17 An example of 12-bit fixed-point number with $fl=4$

Assume there exist three fixed-point numbers A, B and C with fraction length equal to fl_A , fl_B and fl_C . For the multiplication $A \times B = C$, fraction length will be $fl_C = fl_A + fl_B$. For the addition $A + B = C$, the fixed-point number with less fraction length need to left shift to align the decimal point to the other fixed point number, fraction length will be $fl_C = \max(fl_A + fl_A)$. Fig.18 shows an example of fixed-point arithmetic.

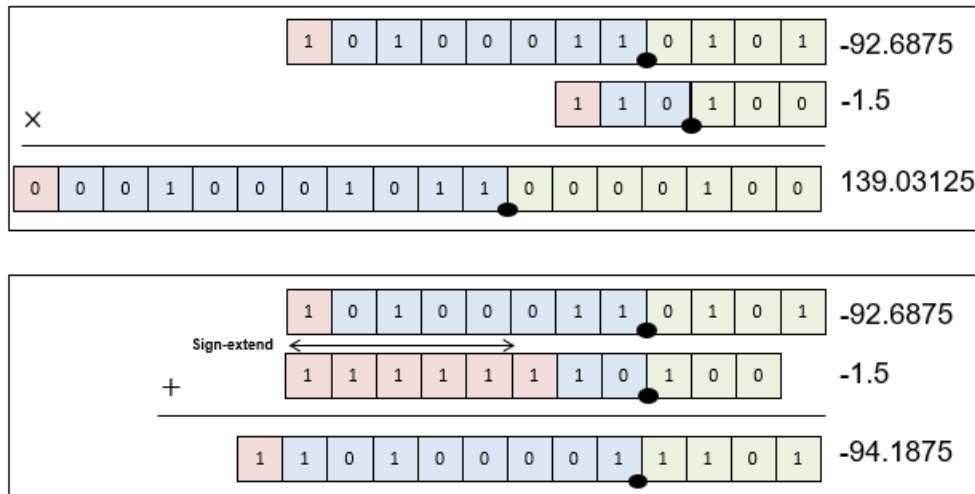


Fig. 18 An example of fixed-point arithmetic

After multiplication and accumulation, we need to quantize the result to 12-bit fixed point with $fl = 4$. Fig.19 illustrates the case that 18-bit accumulated output quantized to 12-bit fixed point.

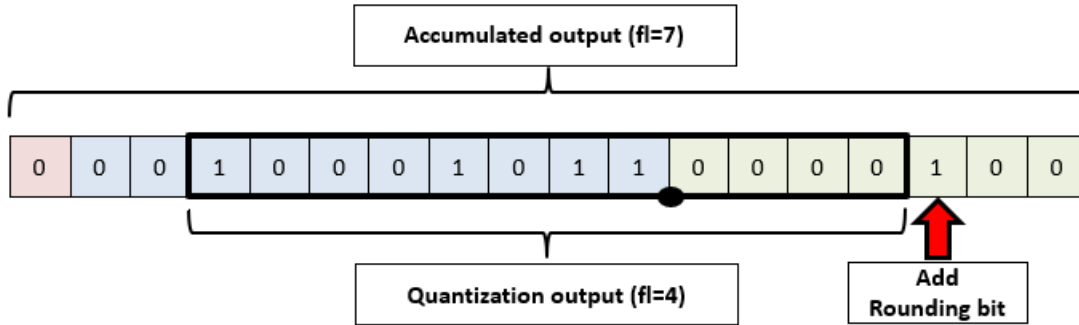


Fig. 19 An illustration of quantizing output feature maps

The following pseudo code may help you to understand fixed-point arithmetic and quantization.

```

accumulated_output = convolved_output + bias << (activation_fl + weight_fl - bias_fl);    Add bias
rounding_output = accumulated_output + 1 << (weight_fl - 1);                          Add rounding bit
quantized_output = rounding_output >>> weight_fl                                     Remove extra fraction length
if (quantized_output > 2047)                                                         Clip to maximum and minimum
    quantized_output = 2047;
else if (quantized_output < -2048)
    quantized_output = -2048;
else
    quantized_output = quantized_output [11:0];

```

The previous example is 12-bit fixed point with $fl = 4$. In this assignment, **all feature maps are represented in 10-bits with $fl = 6$** . For weights and biases, they are represented in 10-bits with $fl = 7$.

Normalization implementation

Since in this assignment, we use fixed-point to represent numbers instead of floating points, μ and MAE should also compute with fix-point. You should allocate an appropriate bit length to ensure accurate storage of these values. **Besides, for fix-pointed division, you only need to perform to $fl = 6$, while the more fractional point can be clipped.** Finally, after affine linear transformation, we also need to quantize the output $y_i = \gamma_i * \hat{x}_i + \beta_i$ to 10-bits with $fl = 6$.

For fixed-point division, we first change the input dividend and divisor to positive numbers, then perform the division to $fl = 6$. If the original answer is negative, we then

change the result back to a negative number (i.e. positive divisor and negative dividend, or negative divisor and positive dividend).

Multi-head self-attention implementation

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{c_{head}}}\right)V$$

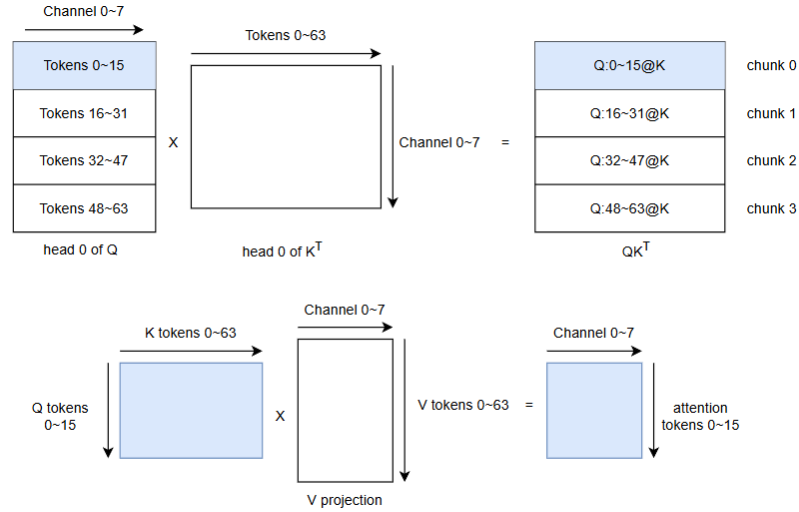
In this assignment, the embedded dimension is 16 and we separate it into 2 heads, each head has $c_{head} = 8$. **Therefore, the scaling factor is $\frac{1}{\sqrt{c_{head}}} = \frac{1}{\sqrt{8}} \approx (0.3515625)_{10} = (0.0101101)_2$. You can simply multiply this value, which is quantized with $fl = 7$.**

Due to the limited SRAM capacity, we cannot perform multi-head self-attention in parallel. Instead, we first compute attention for head 0 and store the output in SRAM Group D, then proceed to compute attention for head 1.

When performing each head's attention, the QK^T matrix has a size of 64×64 in this assignment, which exceeds the storage capacity of SRAM Group D. Therefore, we divide the QK^T matrix into four chunks and process them sequentially — compute the first chunk, store its result, then proceed to the next chunk, and so on.

In short, for each head, the process is as follows (also illustrated in Fig. 21):

1. Compute the Q, K, and V projections for that head. Each projection contains $64 \text{ tokens} \times 8 \text{ channels}$, allowing all data to be stored in SRAM Group C.
2. Perform one chunk-wise QK^T multiplication.
 - Chunk 0: Tokens 0~15 of Q multiply all of tokens of K
 - Chunk 1: Tokens 16~31 of Q multiply all of tokens of K
 - Chunk 2: Tokens 32~47 of Q multiply all of tokens of K
 - Chunk3: Tokens 48~63 of Q multiply all of tokens of K
3. apply softmax and multiply softmax result by V.
4. Proceed to the next chunk and repeat steps 2–3 until all chunks are completed.

Fig. 21 QK^T separate into 4 chunks

We store the chunk (16*64) into SRAM D address 32~63 like Fig.22. and we store the attention output back to SRAM D address 0~31 like Fig. 23.

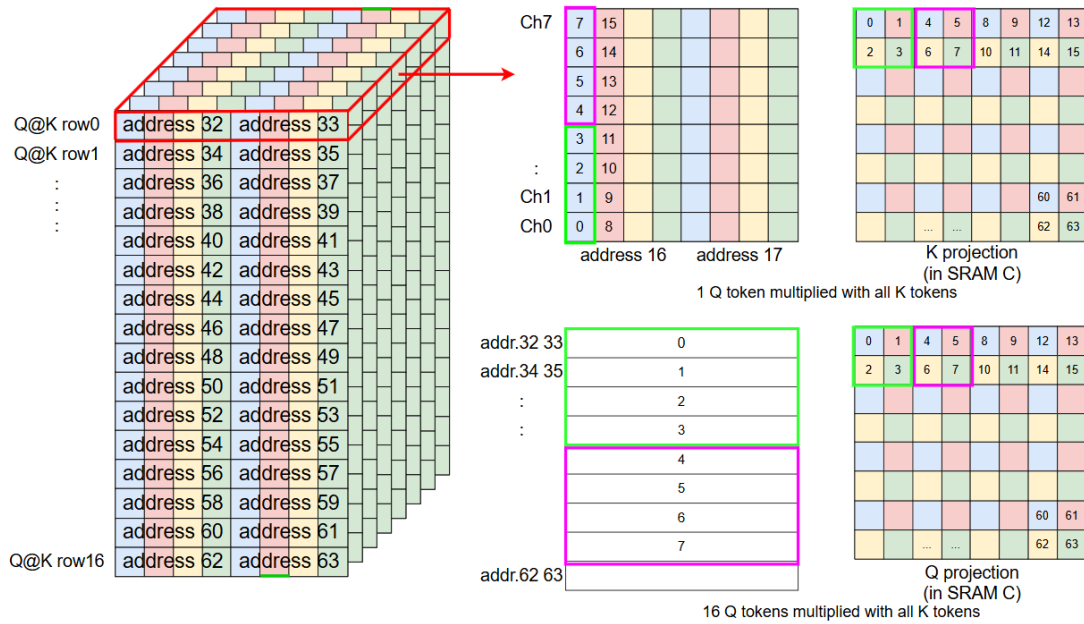


Fig. 22 SRAM mapping for Q@K

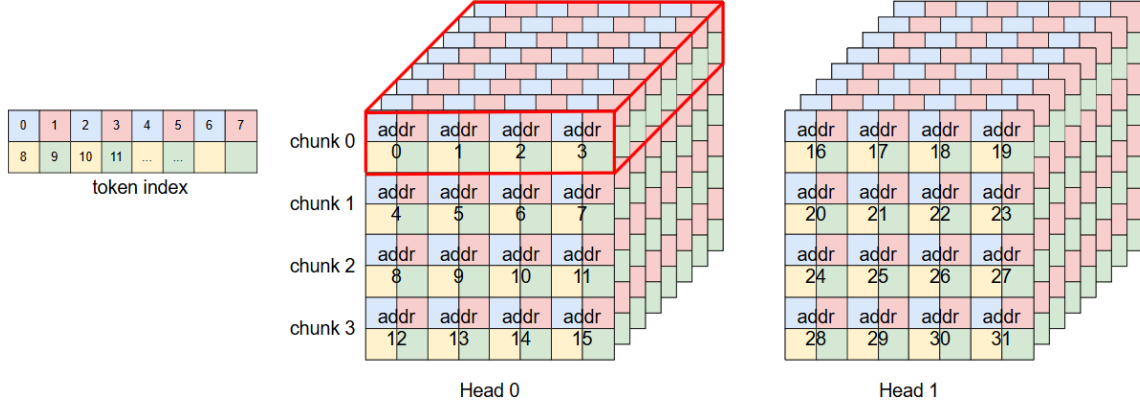


Fig. 23 SRAM mapping for attention (Q, K, V)

Softmax implementation

The softmax function is defined as

$$y_i = \frac{e^{x_i}}{\sum_{n=1}^K e^{x_n}} \text{ for } i = 1, \dots, K$$

Where x is the input vector, i represents the i -th element and K is the length of the vectors.

Implementing softmax in hardware involves several challenges, as it requires implementing exponential functions, summation, and division. For the exponential function, several methods can be employed, such as: (1) Look-up table (LUT) (2) CORDIC (3) Taylor Series.

Here, we introduce the method of using the Taylor series. By starting the expansion at $x = a$, the Taylor series can be expressed as:

$$f(x) = f(a) + \frac{f^{(1)}(a)}{1!} (x - a) + \frac{f^{(2)}(a)}{2!} (x - a)^2 + \frac{f^{(3)}(a)}{3!} (x - a)^3 + \dots$$

where $f^{(n)}(x)$ represents n -th order derivative of $f(x)$. When $f(x) = e^x$, starting the Taylor expansion at $x = a$. we can rewrite it as:

$$f(x) = e^a + \frac{e^a}{1!} (x - a) + \frac{e^a}{2!} (x - a)^2 + \frac{e^a}{3!} (x - a)^3 + \dots$$

For example, to calculating $e^{10.32}$ by expending around $a = 10$, the expression becomes:

$$e^{10.32} = e^{10} + \frac{e^{10}}{1!} (10.32 - 10) + \frac{e^{10}}{2!} (10.32 - 10)^2 + \frac{f(3)(a)}{3!} (10.32 - 10)^3 + \dots$$

When choosing the point of expansion a close enough to the position x where the exponential function is to be calculated, the absolute value of $(x - a)$ is less than 1, and the higher-order terms in the Taylor expansion approach 0. Moreover, the closer x is to a , the faster the convergence rate. Fig. 24 shows a simple flow using LUT and Taylor series expansion to implement exponential function

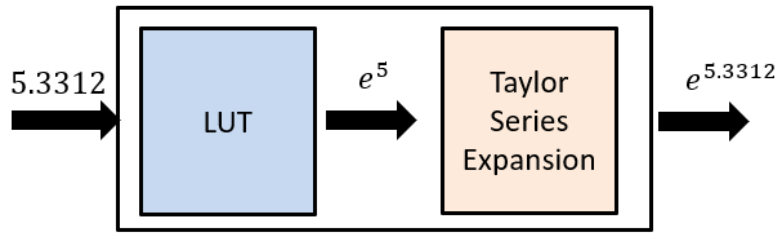


Fig. 24 An illustration of implementation of exponential function

In softmax part, many methods can be implemented, and you are encouraged to explore different approaches. The only specification is that the output of the softmax function must be represented as a 10-bit fixed-point value with fractional length of 6. In other words, you only need to perform fix-pointed division to $fl = 6$, while the more fractional point can be clipped.

We have provided a LUT in the appendix to help you get started. However, you are welcome to create your own LUT if you prefer.

Design Flow

1. Read the patch embedded data from SRAM A.
2. Implement normalization.
3. Quantize the result to 10-bit and write the result to SRAM B.
4. Read normalized data from SRAM B.
5. Implement head 0 of K projection, Q projection and V projection.
6. Quantize the result to 10-bit and write the result to SRAM C.
7. Read K projection and Q projection from SRAM C.
8. Implement multiplication of attention head 0 first 16 rows results.
9. Multiply each result with $1/\sqrt{C_{\text{head}}}$
10. Quantize the result to 10-bits and write the result to SRAM D.

11. Read the attention data from SRAM D.
12. Implement Softmax.
13. Quantize the result to 10-bits and write the result to SRAM B.

14. Read V projection from SRAM C and softmax result from SRAM B.
15. Implement multiplication of attention head 0 first 16 rows results.
16. Quantize the result to 10-bits and write the result to SRAM D.

17. Repeat step 7~15 to finish remaining rows.
18. Repeat step 1~16 to finish head1.

19. Read V softmax multiplication result from SRAM D.
20. Implement O projection.
21. Quantize the result to 10-bit and write the result to SRAM C.

22. Read O projection from SRAM C and patch embedded data from SRAM A.
23. Implement residual add
24. Quantize the result to 10-bit and store the result to SRAM B.

Grading

1. RTL coding simulation: (60%)

If you use your own dataflow, make sure it writes data back to the corresponding SRAM and follows the correct data format for the testbench to verify the results.

 - I. Part 1: Finish QKV head0 linear projection: 20%
(Pass testbench +define+QKV_HEAD0)
 - II. Part 2: Finish SoftMax layer (head0 chunk0 only): 20%
(Pass testbench +define+SOFTMAX_HEAD0_CHUNK0)
 - III. Part 3: Finish MHSA blocks: 15%
(Pass testbench +define+VMUL_HEAD0 and +define+VMUL_HEAD1)
 - IV. Part 4: Finish Residual add: 5%
(Pass testbench +define+RESIDUAL1)
2. Use Spyglass and LEC to show the synthesizability and logic equivalence.(5%)

3. PI ranking: (35%)

(Only for those who finish whole design) Logic synthesis with the provided scripts. Only two script files can be modified: **0_readfile.tcl** for adding your self-defined RTL files, and **synthesis.tcl** for changing the clock timing constraint TEST_CYCLE (e.g. 3.9 for 3.9 ns). Your timing slack should not be negative. The grading is based on your performance index (PI) and computed using a Gaussian-based ranking formula:

$$Grade\ point: \begin{cases} 0\% & \text{if } PI < Threshold \\ 35 * (\mu + \sigma * \Phi^{-1} * (1 - \frac{rank - 0.5}{N})) & \text{if } PI \geq Threshold \end{cases}$$

Where:

Rank = your rank (1 for best performer)

N = number of eligible students in ICLAB

Φ^{-1} = inverse of the standard normal cumulative distribution function

μ and σ = normalization parameters to map ranks into score range, $\mu = 60$, $\sigma = 20$ in this homework.

Definition of the performance index:

A: Total area (e.g. Total area: 227862)

T: TEST_CYCLE (your timing constraint, e.g. 3.5)

C: The cycle counts you completed the simulation, which is shown on the terminal (e.g. Total cycle count = 3712)

Performance index = $A \times T \times C$ (e.g. $2.96 * 10^9$ for the above example)

Bonus: 3% will be given to the best work in terms of the PI, and 1% to the second best.

Deliverable

Please submit all files mentioned below, and organize the file structure as follow:

1. Synthesizable Verilog.
 - ViT_top.v
 - All other (if any) RTL codes for your own defined modules (*.v) in /hdl folder.
2. Command-line arguments file.
 - sim.f (should include all your modules used in your design).
3. A Spyglass pass report file.
 - Spyglass.rpt
4. LEC equivalent report file.

- golden.f (file list of the RTL)
 - lec.log (RTL to NETLIST)
5. Synthesis script and report
- 0_readfile.tcl (should include all RTL file used in your design)
 - synthesis.tcl (should include time constraint of your design)
 - da.log
 - report_time_ViT_top.out
 - report_area_ViT_top.out
 - report_resource_ViT_top.out
 - ViT_top_syn.v
 - ViT_top_syn.sdf
6. misc.txt
- Summarize you're A, T, C, PI and the RTL design stage (PART 1, PART2 or PART3)

Directory	Filename
HW5_{studentID}	1. misc.txt 2. spyglass.rpt
HW5_{studentID}/hdl/	1. ViT_top.v 2. All other your RTL files(*.v)(if any)
HW5_{studentID}/sim/	sim.f
HW5_{studentID}/lec/	1. golden.f 2. lec.log
HW5_{studentID}/syn/	1. .synopsys_dc.setup 2. 0_readfile.tcl 3. synthesis.tcl 4. da.log 5. report_timing_ViT_top.out 6. report_area_ViT_top.out 7. report_resource_ViT_top.out 8. ViT_top_syn.v 9. ViT_top_syn.sdf

Note: If your student ID is u123456789, HW5_{studentID} denotes HW5_123456789. Compress the folder HW5_{studentID} into **HW5_{studentID}.zip** and submit. **10 points punishment for wrong file delivery.**

File Organization

Directory	Filename	Description
./	1. misc.txt 2. Softmax_TB.xlsx	1. Record the performance of your design 2. Softmax table for reference
hdl/	ViT_top.v	Top module
sim/	1. display.py 2. sim.f	1. Program for you to see all layer outputs

	3. test_top.v 4. run_sim.sh 5. set_env.sh 6. clean.sh	2. File list for RTL simulation 3. Testbench 4. pre-sim script 5. Python environment for display.py 6. Script for cleanup simulation result
sim/golden	*.dat	Golden result
sim/feature	*.npv	Python file for check answer
sim/param	*.dat	Parameter for model
sim/sram_model	*.v	SRAM simulation model
syn/	1. .synopsys_dc.setup 2. *.tcl	1. DC environment file 2. tcl file for DC scripts
lec/	1. *.do 2. run_lec.sh 3. revise.f 4. golden.f	1. do file for LEC script 2. shell script for lec 3. File list for netlist 4. File list for the RTL

Simulation Tools

- For debugging, we have provided display.py for you to check all layers' output feature maps.
 - Log in EE workstation wsXX. (Notice that some host may not include python3.6)
 - Type source set_env.sh on the terminal at the sim directory.
 - Type python3.6 display.py on the terminal. After entering layer and channel numbers, it will print the corresponding results.
- All RTLs should be synthesizable. i.e. your RTL code should pass Spyglass examination before submitting
- In run_sim.sh, there are several configurations that can be quickly modified or commented out to test different layers. Adjust these settings to select your desired configuration. All patterns (0–9) must pass the simulation.

Configuration	Description
{LAYER}	Select the testing layer when the valid signal goes high
PAT_L	Starting pattern number (must be between 0 and 9)
PAT_U	Ending pattern number (must be between 0 and 9)
FLAG_VERBOSE	Generate detailed simulation reports
FLAG_DUMPWV	Dump the FSDB waveform
FLAG_SHOWNUM	Display digit classification information
END_CYCLES	Prevent dead loops

Appendix

Exponential Table

We have provided Softmax_TB.xlsx with a 10-bit fixed-point input (fractional length 6) and a 100-bit fixed-point output (fractional length = 50). The provided table is for reference only. You can create your own table or modify the one provided.

Name	Description
Fixed-point Input	Integer format of input
Input value	Real value of input 10-bits fixed-point with fl = 6
Exponential Value	Corresponding exponential value (Using mpmath with dps =100)
Exponential Quantized value	Quantized value of exponential 100-bits fixed-point with fl = 50
Fixed-point Output	Integer format of output

Reference

[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, “An image is worth 16x16 words: transformer for image recognition at scale”

[2] Y. Lin¹, Y. Li, T. Liu, T. Xiao, T. Liu and J. Zhu Towards, “Fully 8-bit Integer Inference for the Transformer Model”